NEBULA SUPERMARKET OPS AGENT

An AI-powered supermarket operations assistant designed to help a kirana/supermarket store manage day-to-day operations through a conversational Telegram interface.

The agent can assist with inventory management, stock tracking, billing, sales information, customer credit/khata operations, reports, invoice generation, and other supermarket operations.

Project Overview

Small supermarket and kirana store owners often manage inventory, sales, customer credit, and daily reports manually.

The Nebula Supermarket Ops Agent provides a conversational interface where store operators can interact with their business data using natural language.

Instead of navigating multiple menus or manually querying a database, the user can simply send a message through Telegram.

Example

User:

Show me the current stock

Agent:

[Retrieves inventory information from the database]

The AI agent understands the request and uses the appropriate backend tool to perform the operation.

Key Features

Inventory Management

- Add and manage products
- Check current stock
- Receive new stock
- Identify low-stock products
- Track inventory changes

Billing & Sales

- Create bills
- Add products to bills
- Modify bill items
- Finalize bills
- Calculate totals
- Track sales

Customer & Khata Management

- Maintain customer information
- Track customer credit
- Record payments
- Check outstanding balances

Reports

- Daily sales reports
- Daily closing information
- Inventory reports
- Low-stock reports
- Business analysis

Invoice Generation

- Generate invoices for finalized bills
- Provide invoice documents through the Telegram bot

AI Conversational Interface

Users can interact with the system using natural language instead of learning complicated commands.

Telegram Integration

The supermarket agent is accessible through Telegram, making it convenient for store operators to use from a mobile phone.

Persistent Preferences

The system supports storing standing preferences such as:

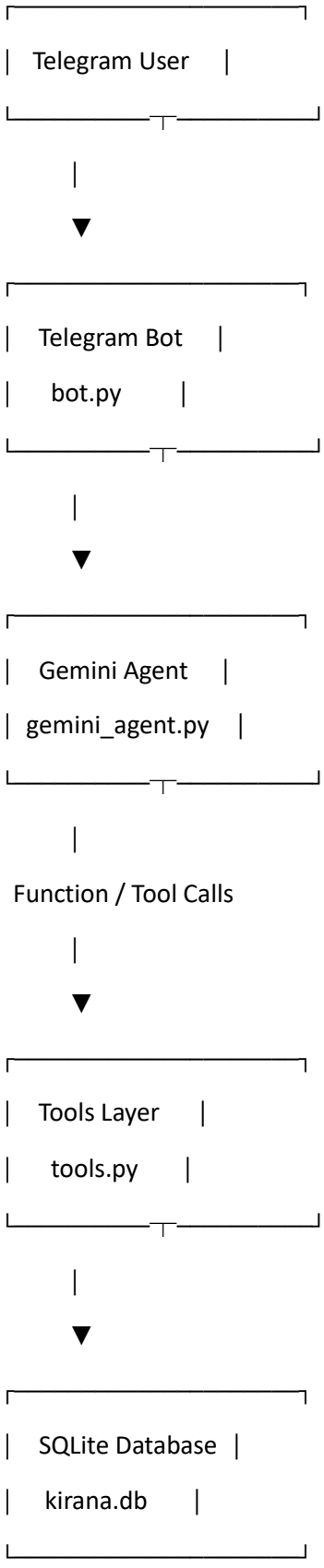
- Preferred brands
- Payment preferences
- Store information

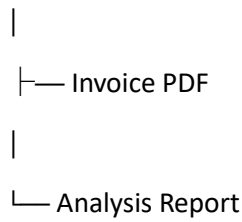
These preferences can persist across conversations and application restarts.

Idempotent Operations

Important operations use idempotency mechanisms to reduce the risk of duplicate transactions when Telegram updates are redelivered or requests are repeated.

System Architecture

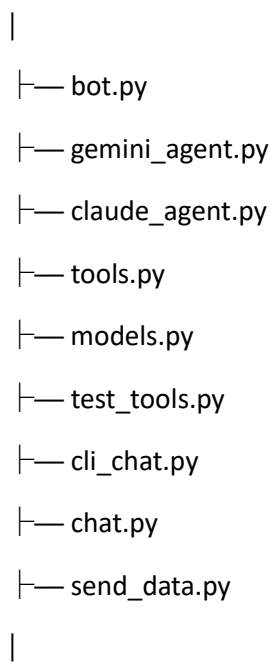




Technologies Used

Technology	Purpose
Python	Backend development
Google Gemini API	AI agent / natural language understanding
python-telegram-bot	Telegram integration
SQLite	Local database
SQLAlchemy	Database interaction
ReportLab	Invoice PDF generation
Python-dotenv	Environment variable management
Git & GitHub	Version control and project hosting

Project Structure



├── documents/

│   ├── invoice.py

│   └── deck.py

│

├── requirements.txt

├── README.md

├── .gitignore

│

└── .env

Installation

1. Clone the repository

git clone YOUR_GITHUB_REPOSITORY_URL

Move into the project directory:

cd kirana-agent

2. Create a virtual environment

python -m venv venv

Activate it on Windows:

venv\Scripts\activate

3. Install dependencies

pip install -r requirements.txt

Environment Configuration

Create a .env file in the project root.

ANTHROPIC_API_KEY=your_anthropic_api_key

GEMINI_API_KEY=your_gemini_api_key

TELEGRAM_BOT_TOKEN=your_telegram_bot_token

DB_PATH=kirana.db

SHOP_NAME=Nebula Kirana Store

SHOP_GSTIN=your_gstin

SHOP_ADDRESS=your_shop_address

Security

Never commit .env to GitHub.

The .gitignore file should include:

.env

*.db

__pycache__/

venv/

API keys and bot tokens must remain private.

Running the Application

Activate the virtual environment:

venv\Scripts\activate

Run the Telegram bot:

python bot.py

You should see:

Bot starting (polling)...

Application started

Open the configured Telegram bot and send a message.

Example:

hello

The agent will respond conversationally.

Example Interactions

General Query

User:

Hello

Agent:

Hello! How can I help you with Nebula Kirana Store today?

Inventory

User:

Show me the current stock

The agent retrieves the inventory information using the backend tools.

Sales

User:

Show today's sales

The agent retrieves the relevant sales information from the database.

Low Stock

User:

Which products are low in stock?

The agent checks inventory levels and reports the relevant products.

Bill Processing Flow

The billing process follows a controlled workflow:

Start Bill

↓

Add Items

↓

Edit / Update Items

↓

Calculate Total

↓

Finalize Bill

↓

Update Stock

↓

Generate Invoice

Stock is updated only when the bill is finalized, helping prevent incorrect inventory deductions during an unfinished transaction.

Database

The project uses **SQLite** for local data storage.

The database is used to maintain information related to:

- Products
- Inventory
- Bills
- Sales
- Customers
- Credit / Khata

- Payments
- Preferences
- Processed Telegram updates

Duplicate Update Protection

Telegram may redeliver updates during network interruptions.

The application stores processed Telegram update_id values to prevent the same update from being processed multiple times.

This provides an additional idempotency layer for Telegram message processing.

Invoice Generation

After a bill is finalized, the system can generate an invoice PDF.

The invoice functionality is implemented in:

documents/invoice.py

The generated document can be sent back to the user through Telegram.

Business Analysis

The project also includes functionality for generating business analysis/reporting outputs.

The analysis functionality is implemented in:

documents/deck.py

This can be used to present useful supermarket operational insights.

Testing

Tool-level functionality can be tested using:

python test_tools.py

This allows backend functionality to be tested independently from the Telegram interface.

Security Considerations

- API credentials are stored in environment variables.
- .env is excluded from Git.
- Telegram bot tokens are kept private.
- Database files can be excluded from version control when appropriate.
- Important transactional operations use idempotency keys.
- User conversation history is maintained separately from persistent store preferences.

Demo

Demo Video

The demo demonstrates the Telegram-based supermarket agent and its interaction with the supermarket operations system.

Project Objectives

The main objectives of this project are:

- Automate common supermarket operations.
- Provide a natural-language interface for store operators.
- Reduce manual inventory and billing work.
- Maintain reliable transaction processing.
- Provide useful sales and inventory insights.
- Make supermarket operations accessible through Telegram.

Future Enhancements

Potential future improvements include:

- Web-based supermarket dashboard
- Real-time analytics
- Barcode scanner integration
- Voice-based supermarket operations
- Multi-store management
- Cloud database support
- Automated stock-reorder suggestions
- Advanced sales forecasting
- Customer purchase recommendations
- Role-based access control
- Multi-language support

Project

Project: Nebula Supermarket Ops Agent

Bot Name: Abi Bot

Telegram Search: @Abinaya510250

Store: Nebula Kirana Store

Interface: Telegram

Backend: Python

AI: Google Gemini

Database: SQLite

Conclusion

The **Nebula Supermarket Ops Agent** demonstrates how an AI-powered conversational agent can be integrated with real business operations.

By combining natural-language interaction, tool-based execution, database persistence, transaction safety, reporting, and Telegram integration, the system provides a practical AI assistant for supermarket and kirana store operations.